COMP 2233: Data Structures

Chapter 7: Linked Lists

Assoc.Prof.Dr. Ömer ÇETİN

Problem Solving Pipeline with Computer

Problem Solving Pipeline

1. Problem Statement: Real-world computational requirements.

2. Algorithm Development (Human): Logic & structure design.

3. Program Development (Human): Writing C/C++ source code.

4. Program Translation (Software): Compiler translating to binary.

5. Program Execution (Hardware): Executing in dynamic RAM.

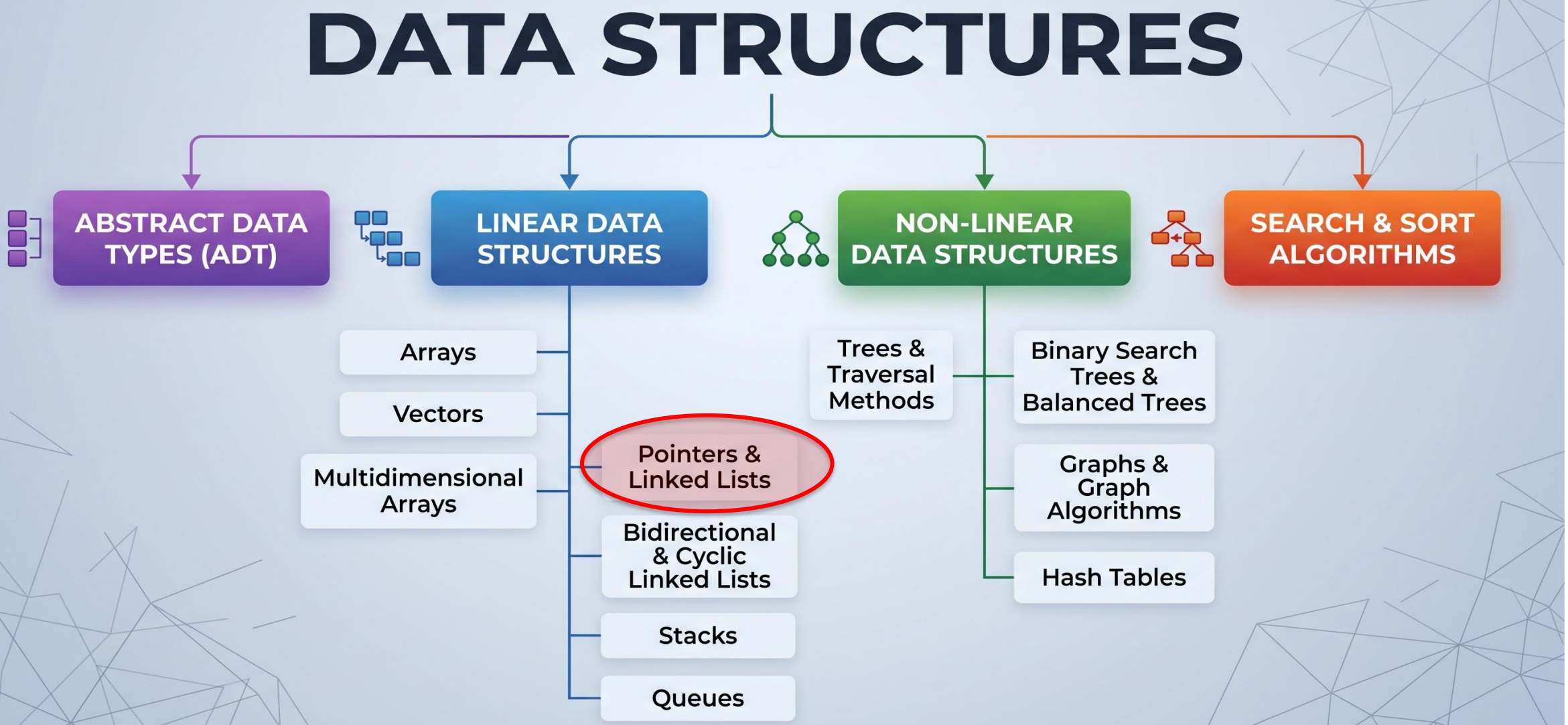
Linked Lists in Memory Execution

- **Dynamic Node Allocation:** Linked lists allocate nodes dynamically on the Heap during execution.
- **Non-Contiguous Advantage:** Avoids requiring large unbroken memory blocks.
- **Error Management:**

  - Syntax Errors: Caught during compilation translation.

  - Semantic Errors: Null pointer dereferences & memory leaks handled at execution time.

Course Content



Syllabus Map: Focus on Linked Lists

First-Half Topics

1. Introduction
2. Algorithm Analysis & Complexity
3. Data Structures & Data Models
4. Single Dimension Arrays & Vectors
5. Multi-Dimension Arrays
6. Pointers
-> 7. Linked Lists (Singly Linked Lists)

Advanced List & Data Topics

-> 8. Doubly Linked Lists
-> 9. Circular Linked Lists
10. Mid-Term Exam
11. Queues & Stacks
12. Trees & Binary Search Trees
13. Graphs
14. Hash Tables
15. Search & Sort Algorithms

Introduction to Linked Lists

What is a Linked List?

- **Linear Data Structure:** Elements (called nodes) form a sequential chain.
- **Non-Contiguous Allocation:** Unlike arrays, nodes do NOT occupy contiguous memory locations.
- **Node Components:** Each node holds data and a pointer/reference to the next node in sequence.
- **Dynamic Resizing:** Can grow or shrink indefinitely at runtime based on available RAM.

Key Operational Advantages

- **O(1) Constant-Time Insertion/Deletion:** Inserting or removing nodes at known pointers requires simple link updates.
- **Zero Element Shifting:** Eliminates the costly O(N) element shifting required by static arrays.
- **Dynamic Memory Efficiency:** Allocates exact memory needed per element without pre-allocated buffer overhead.

Core Concepts: Node, Head, and Link Anatomy

3 Key Architectural Components

1. Node: The basic atomic unit of a linked list containing data payload and pointer link.

2. Head: The mandatory starting pointer variable that stores the memory address of the first node.

3. Pointer / Link: A reference attribute storing the RAM location of the next node in the chain.

Node Physical Anatomy Diagram

Node Structure:



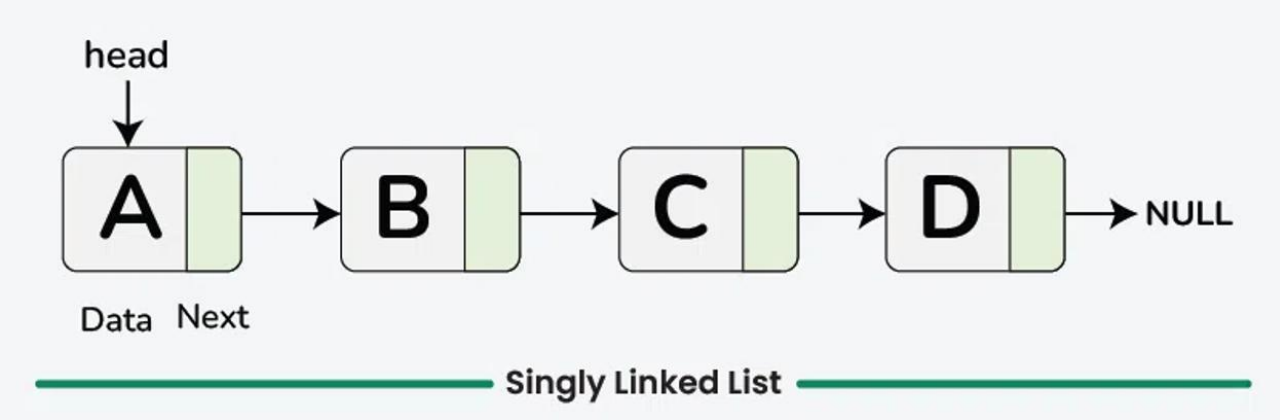
- **Data Section:** Stores integers, floats, structs, or objects.
- **Link Section:** Holds memory address of next node (or NULL if tail).

Types of Linked Lists (1/3): Singly Linked List

Singly Linked List Architecture

- **One-Way Traversal:** Each node contains a single 'next' pointer referencing the subsequent node.
- **Traversal Direction:** Unidirectional movement strictly from Head to Tail (cannot move backwards).
- **Termination:** The final node's next pointer is set to NULL / nullptr.
- **Visual Representation:**

head -> [Data: A | Next] -> [Data: B | Next] -> [Data: C | Next] -> [Data: D | NULL]

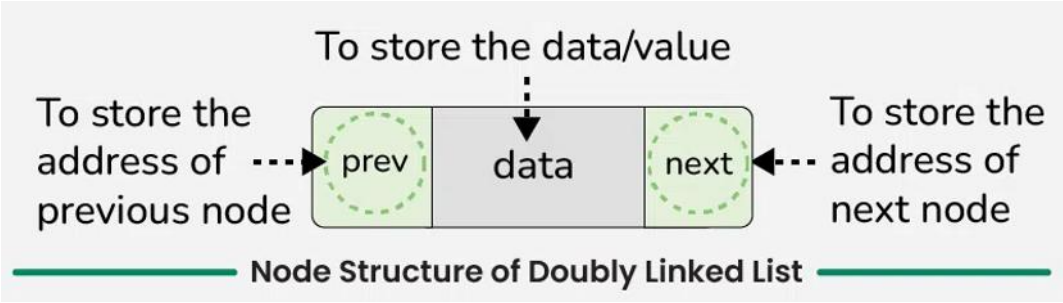


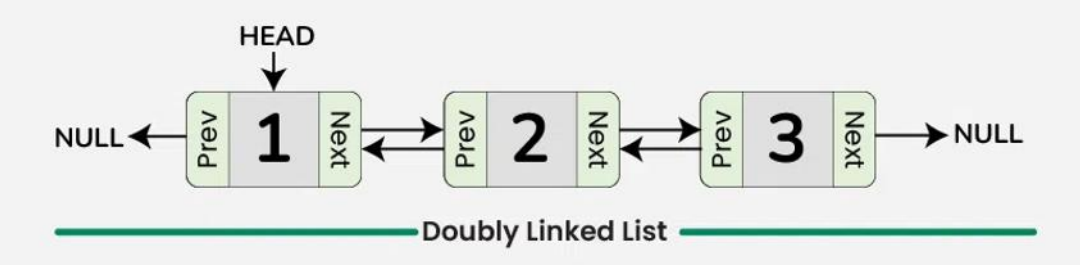
Types of Linked Lists (2/3): Doubly Linked List

Doubly Linked List Architecture

- **Two-Way Traversal:** Each node contains TWO pointer links: 'next' and 'previous' (prev).
- **Node Anatomy:** [Prev Pointer | Data Field | Next Pointer]
- **Bidirectional Navigation:** Allows traversing forward to tail OR backward to head.
- **Visual Representation:**

NULL <- [Prev | 1 | Next] <==> [Prev | 2 | Next] <==> [Prev | 3 | Next] -> NULL





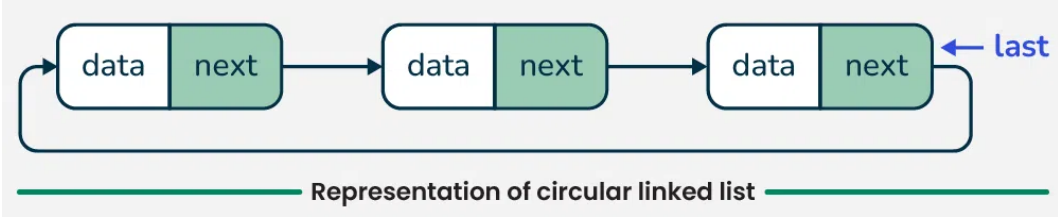
Types of Linked Lists (3/3): Circular Linked List

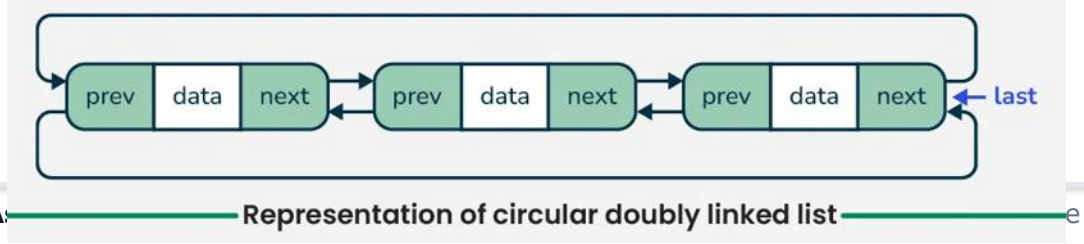
Circular Linked List Architecture

- **Endless Ring Structure:** The last node's 'next' pointer points back to the Head node instead of NULL.
- **Circular Variations:** Circular Singly Linked Lists & Circular Doubly Linked Lists.
- **Use Cases:** Round-Robin CPU process scheduling, media player looping, buffer queues.
- **Visual Representation:**

```
[ Node A ] ---> [ Node B ] ---> [ Node C ]
    ^                                |
    +--------------------------------+ (Last node links back to Head!)
```







Linked List vs. Array Trade-Off Matrix

Feature Dimension	Static Array	Singly Linked List
Memory Allocation	Contiguous RAM block	Non-contiguous scattered nodes
Insert / Delete Time	Expensive O(N) due to shifting	Efficient O(1) via pointer relinking
Element Access Time	Constant time O(1) direct indexing	Linear time O(N) sequential traversal
Capacity Limits	Fixed upon declaration	Dynamic runtime growth
Memory Overhead	Zero pointer overhead	Extra pointer per node (4/8 bytes)

Overview of Basic Operations

1. Insertion & Deletion Ops

- **Insertion Scenarios:**
  a) At the Beginning (New Head)
  b) At the End (New Tail)
  c) At a Specific Position (Middle)

- **Deletion Scenarios:**
  a) At the Beginning (Remove Head)
  b) At the End (Remove Tail)
  c) At a Specific Position (Remove Node)

2. Traversal, Search & Reversal

- **Traversal**: Visiting every node sequentially from Head to Tail.
- **Searching**: Linear search checking node payload values.
- **Reversal**: Reversing node pointers in-place.
- **Additional Utility Ops:**
  - Empty List Check (bos_mu)
  - Purging Entire List (liste_sil)
  - Counting Size

Singly Linked List Mechanics & Implementation

Deep Dive into Node Structures, Memory Maps, and Pointer Re-linking

Node Structure Definition in C (`struct Node`)

Self-Referential Struct Definition

```c
// C Node Structure Definition
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;           // Data payload
    struct Node *next; // Self-referential pointer
};

typedef struct Node Node;

// Usage in function
Node* create_node(int val) {
    Node *new_node = (Node*)malloc(sizeof(Node));
    new_node->data = val;
    new_node->next = NULL;
    return new_node;
}
```

Structural Explanation

- **Self-Referential Pointer:** 'struct Node *next;' holds the address of another struct Node.
- **Data Payload:** 'int data;' stores the node value.
- **Dynamic Instantiation:** 'malloc(sizeof(Node))' allocates node memory on Heap RAM.
- **Arrow Operator (->):** Used to access struct members via pointer ('new_node->data = val;').

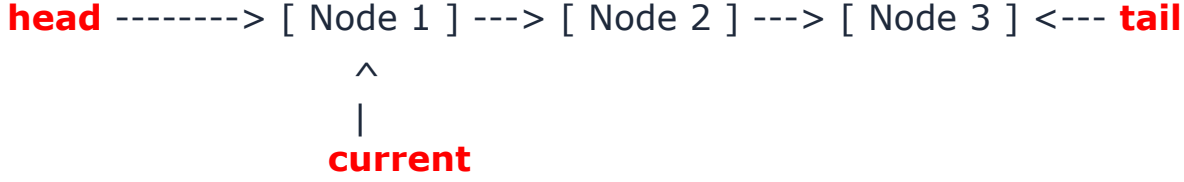
Head, Tail, and Current Element Concepts

Pointer Roles & States

- **Head Pointer:** Mandatory global/local pointer holding address of the FIRST node.
- **Tail Pointer:** Optional pointer holding address of the LAST node (its next == NULL).
- **Current Pointer:** Temporary traversal pointer pointing to the active node being processed.
- **Memory State:** Nodes created dynamically via malloc/new live on the Heap.

Pointer Architecture Diagram

State Diagram:

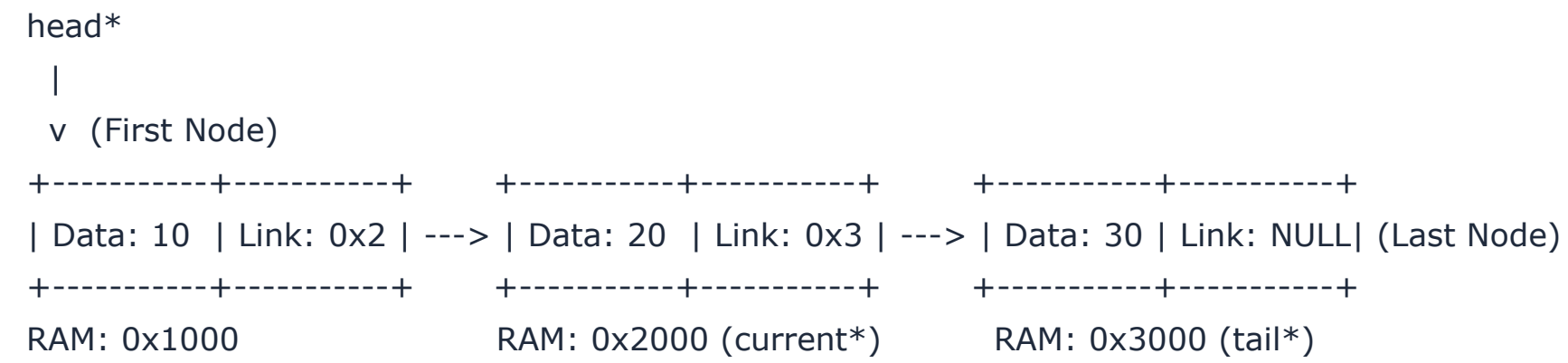


- Head is mandatory to enter the list.
- Tail pointer enables O(1) end appends.
- Current pointer navigates through nodes.

Singly Linked List Memory Map & Pointers

Detailed Pointer Chain Diagram

Memory Map Visualization:



Key Rules:

- head* stores 0x1000.
- Node at 0x1000 holds Link 0x2000.
- Node at 0x3000 holds Link NULL (marks end of list!).

Singly Linked List: One-Way Movement Rule

Unidirectional Traversal Constraint

- **Unidirectional Links:** Düğümler sadece ileriye doğru 'next' işaretçisine sahiptir.
- **Rule:** Movement is strictly ONE-WAY from Head to Tail.
- **Consequence:** Once you move from Node A to Node B, you CANNOT move back to Node A without re-traversing from Head!
- **Remedy:** If bidirectional movement is required by application, upgrade to Doubly Linked List!

Empty Linked List Representation

Empty State Mechanics

- **Empty List Definition:** A linked list containing zero nodes.
- **Pointer State:** Represented simply by 'head = NULL;' (or 'nullptr' in C++).
- **Diagram:**

  head ----> NULL

- **Safety Check:**

  if (head == NULL) { printf("List is empty!\n"); }

  Always verify head != NULL before attempting any node operations!

C Example: Manual Node Creation & Linking

Source Code

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node on the heap
struct Node* createNode(int value) {
    struct Node* newNode = (struct
Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```

```c
int main(void) {
    // Manual node creation and linking
    struct Node* head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(30);

    // Free allocated memory to prevent memory leaks
    struct Node* current = head;
    while (current != NULL) {
        struct Node* temp = current;
        current = current->next;
        free(temp);
    }

    return 0;
}
```

C++ Example: Manual Node Creation & Linking

Source Code

```
// C++ Manual Node Creation & Linking
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* createNode(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = nullptr;
    return newNode;
}

int main() {
    Node* head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(30);
    return 0;
}
```

Execution Output & Chain

Resulting Chain in RAM:

head -> [10 | next] -> [20 | next] -> [30 | nullptr]

Step-by-Step Code Trace:

1. createNode(10): Allocates node with data=10, next=nullptr.
2. head->next = createNode(20): Links node 10 to node 20.
3. head->next->next = createNode(30): Links node 20 to node 30.

Breakdown of Node Creation Code Example

1. Node Structure

- Encapsulates 'int data' payload.
- Encapsulates 'Node* next' self-referential link.

2. createNode Function

- Allocates Heap RAM via 'new Node()'.
- Initializes data and sets next to nullptr.

3. Building the Linked List

- Head holds start node address.
- Subsequent nodes chained via next pointer assignments.

4. Traversing the List

- Uses temporary 'temp' pointer.
- Advances via 'temp = temp->next' until temp == nullptr.

Basic Operations: 1. Insertion at Beginning

Insertion Scenarios Overview

1. Insertion at Beginning (HEAD) <-- CURRENT FOCUS

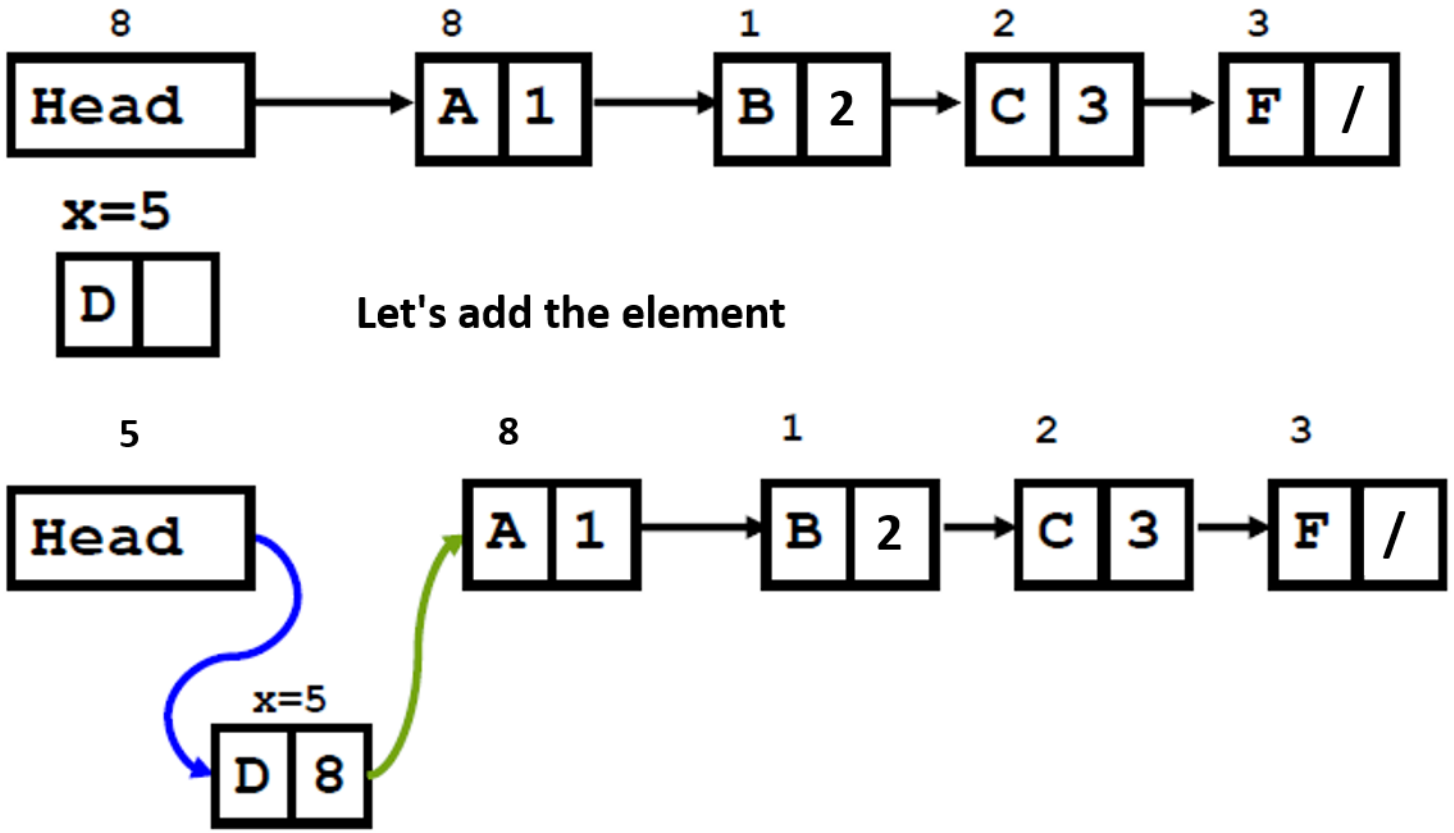
2. Insertion at End (TAIL)

3. Insertion at Specific Position (MIDDLE)

Key Principle for Head Insertion:

- The newly created node will become the NEW HEAD node of the linked list.
- Time Complexity: O(1) constant time! (No list traversal required!).

Insertion at Head



Adding Element to Head: Algorithm & Steps

Algorithm Steps

Step 1: Allocate memory for new node (temp).

Step 2: Assign data value to temp->data.

Step 3: Set temp->next = head; (Connect new node to current head).

Step 4: Set head = temp; (Update head to point to new node!).

CRITICAL ORDER RULE:

You MUST execute 'temp->next = head' BEFORE 'head = temp', otherwise you lose the rest of the list!

Re-linking Pointer Diagram

Pointer Update Diagram:

```
[ New Node: temp ]
      |    (1. temp->next = head)
      v
[ Old Head ] ---> [ Node B ] ---> [ Node C ]
      ^
      |    (2. head = temp)
head*
```

Insertion at Head: Visual Step-by-Step Trace

Step-by-Step Memory Re-linking

Initial List: head -> [1 | b] -> [88 | null]

Adding Node 'temp' with value 'value':

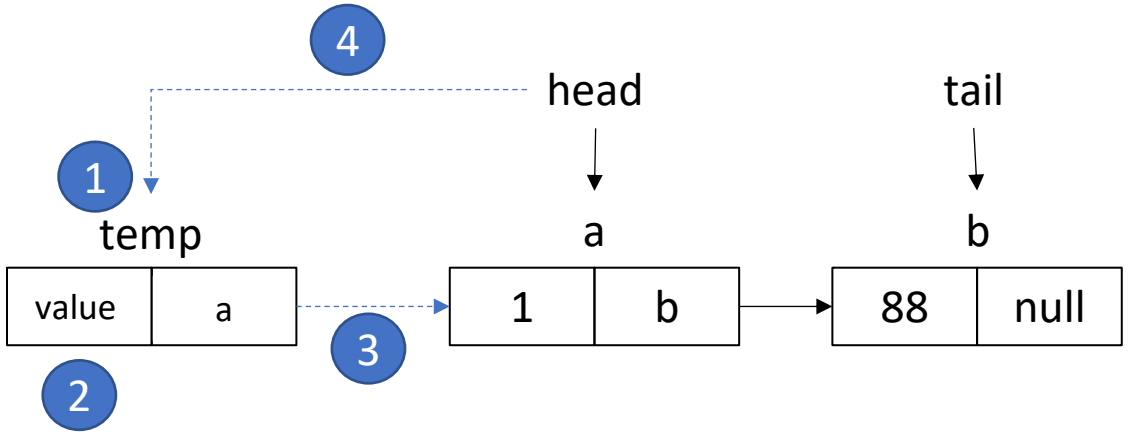
Step 1: Create temp pointer -> temp = new Node(value)

Step 2: Set data payload -> temp->data = value

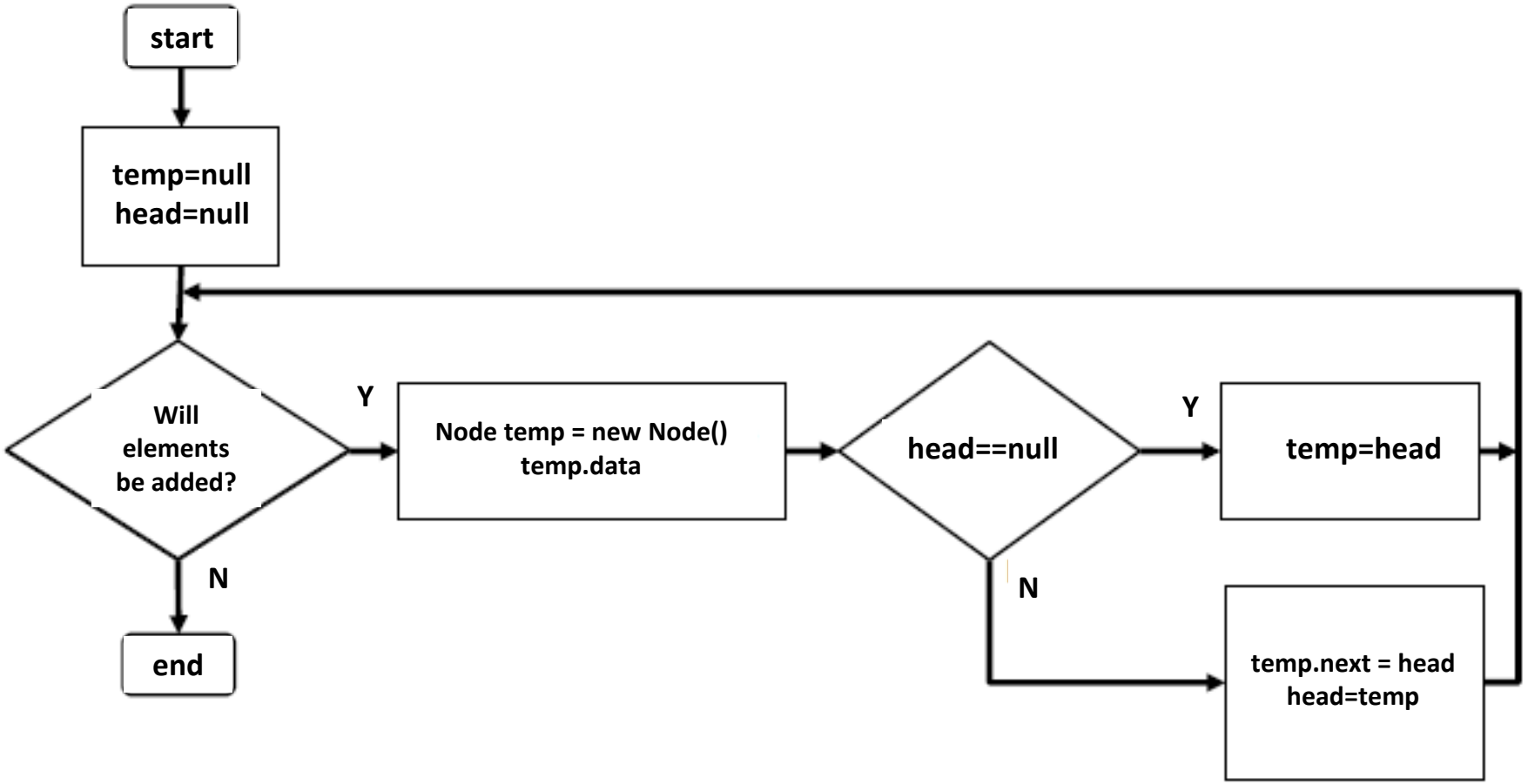
Step 3: Point temp->next to current head 'a' -> temp->next = head

Step 4: Update head pointer to point to temp -> head = temp

Final List: head -> [value | a] -> [1 | b] -> [88 | null]



Insertion at Head: Algorithm



Basic Operations: 1. Insertion at End (Tail)

Insertion Scenarios Overview

1. Insertion at Beginning (HEAD)
2. **Insertion at End (TAIL) <-- CURRENT FOCUS**
3. Insertion at Specific Position (MIDDLE)

Key Principle for Tail Insertion:

- The newly added element will become the NEW TAIL node.
- Traversal: If no tail pointer exists, we must traverse from Head to Tail in O(N) time.
- If a tail pointer exists, tail insertion executes in O(1) time!

Adding Element to Tail: Algorithm & Steps

Algorithm Steps

Step 1: Allocate memory for new node (temp).

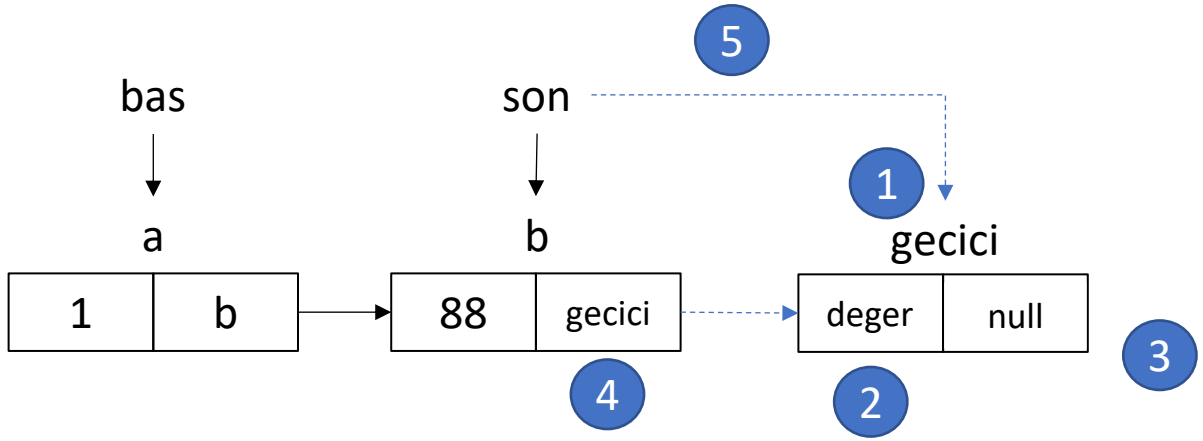
Step 2: Assign data value and set temp->next = NULL.

Step 3: If list is empty (head == NULL), set head = temp and tail = temp.

Step 4: If list is not empty:

tail->next = temp; (Link current tail to temp)

tail = temp; (Update tail pointer to temp!)



Insertion at Tail: Visual Step-by-Step Trace

Step-by-Step Memory Re-linking

Initial List: bas (head) -> [1 | b] -> [88 | null] (son/tail)

Adding Node 'gecici' with value 'deger':

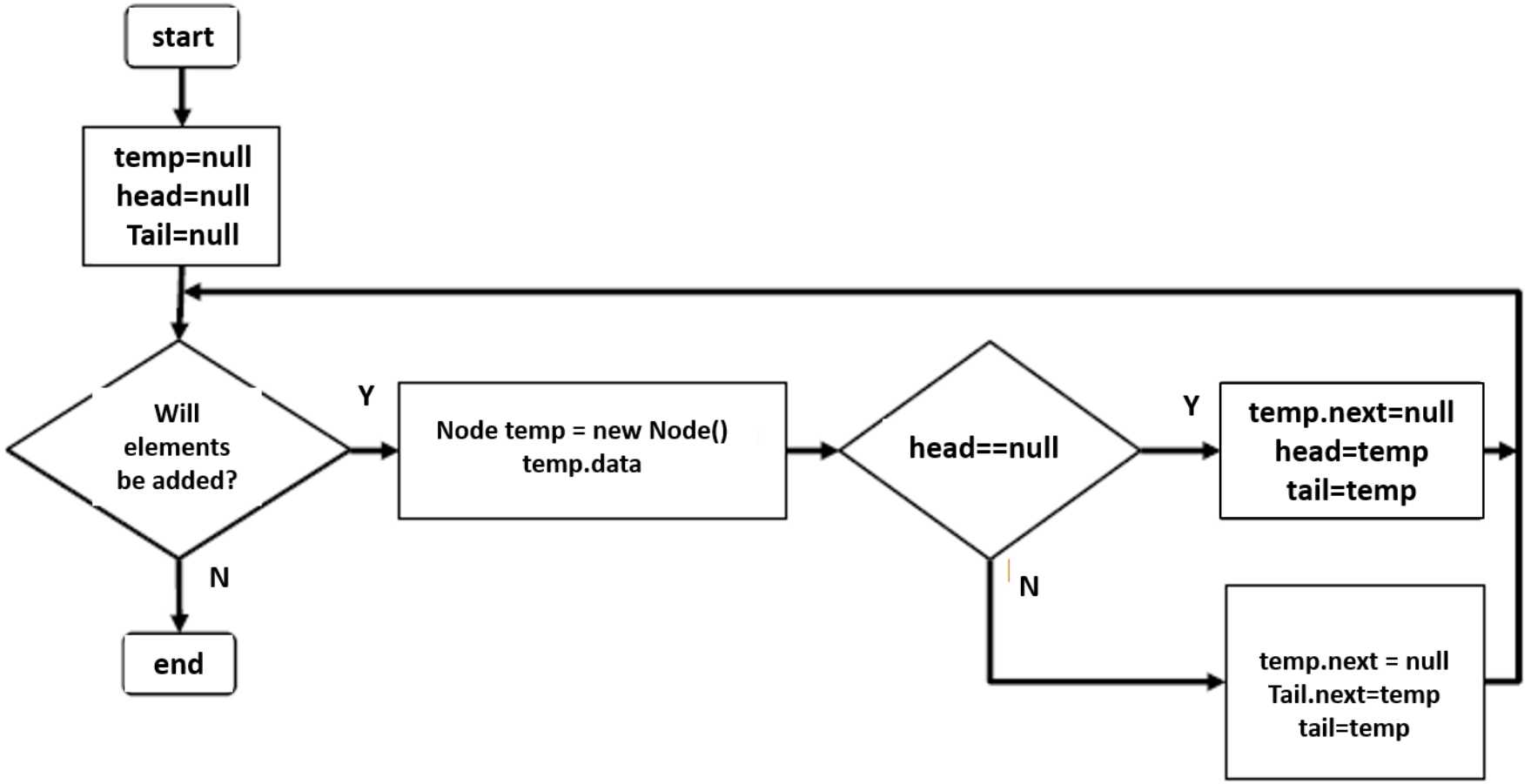
Step 1: Allocate gecici -> gecici->data = deger; gecici->next = null;

Step 2: Re-link current tail 'b' to point to gecici -> son->next = gecici;

Step 3: Update tail pointer to gecici -> son = gecici;

Final List: bas -> [1 | b] -> [88 | gecici] -> [deger | null] (son)

Insertion at Tail: Algorithmic Flowchart



Basic Operations: 1. Insertion at Specific Position

Insertion Scenarios Overview

1. Insertion at Beginning (HEAD)
2. Insertion at End (TAIL)
3. **Insertion at Specific Position (MIDDLE) <-- CURRENT FOCUS**

Key Principle for Middle Insertion:

- Node is inserted into a specific index or sorted order between 'previous' and 'current' nodes.
- Requires searching/traversing to target position in O(N) time.
- Link updates execute in O(1) time once position is reached.

Inserting at Specific Position: Algorithm & Steps

Algorithm Steps

Step 1: Traverse list to find target position, tracking 'previous' and 'current' pointers.

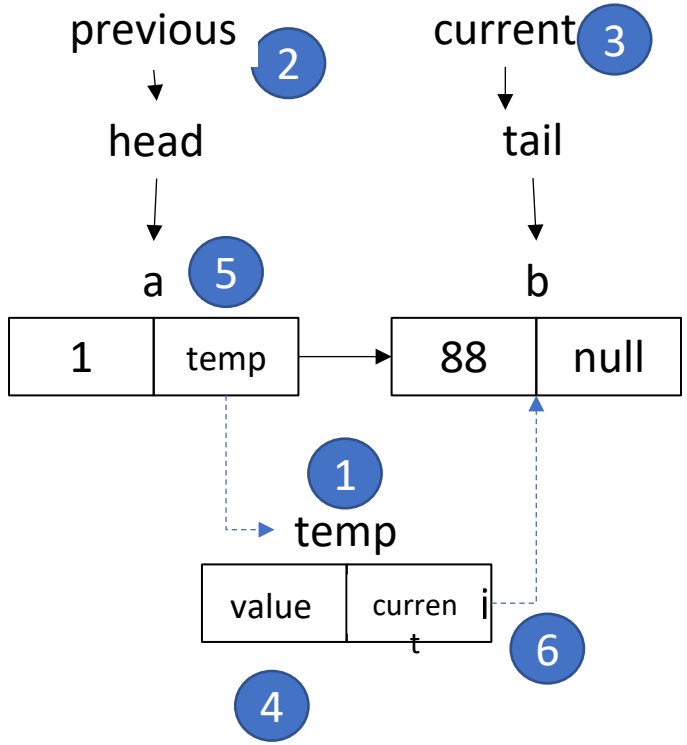
Step 2: Allocate memory for new node (temp).

Step 3: Set temp->next = current; (Connect new node to downstream node).

Step 4: Set previous->next = temp; (Connect upstream node to new node!).

CRITICAL ORDER RULE:

Connect 'temp->next = current' FIRST before modifying 'previous->next'!



Insertion at Middle: Visual Step-by-Step Trace

Inserting Node 'E' with Value 4 between C and F

Initial List: Head -> [A | 1] -> [B | 2] -> [C | 3] -> [F | 0]

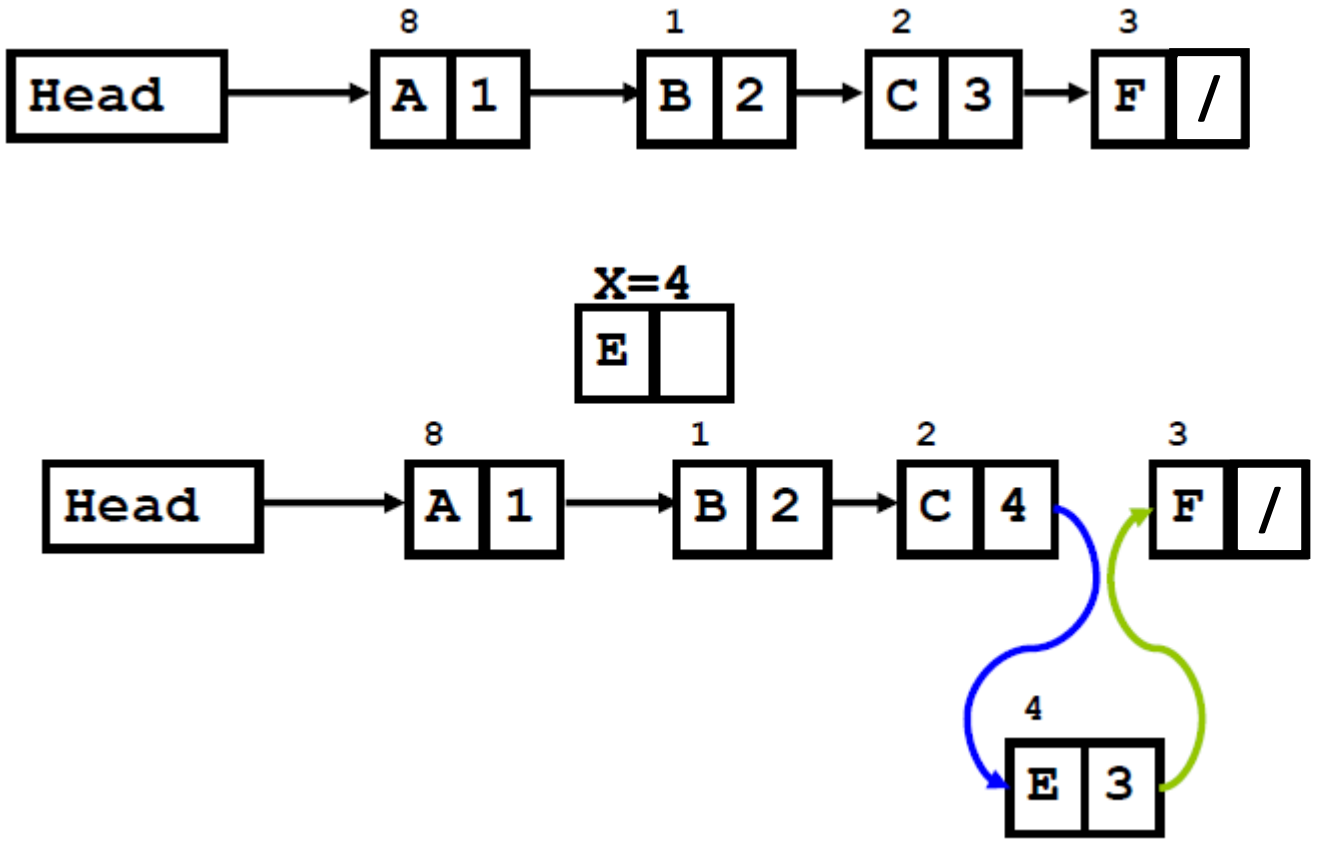
Goal: Insert Node 'E' (value 4) between C and F:

Step 1: Traverse -> 'previous' points to C, 'current' points to F.

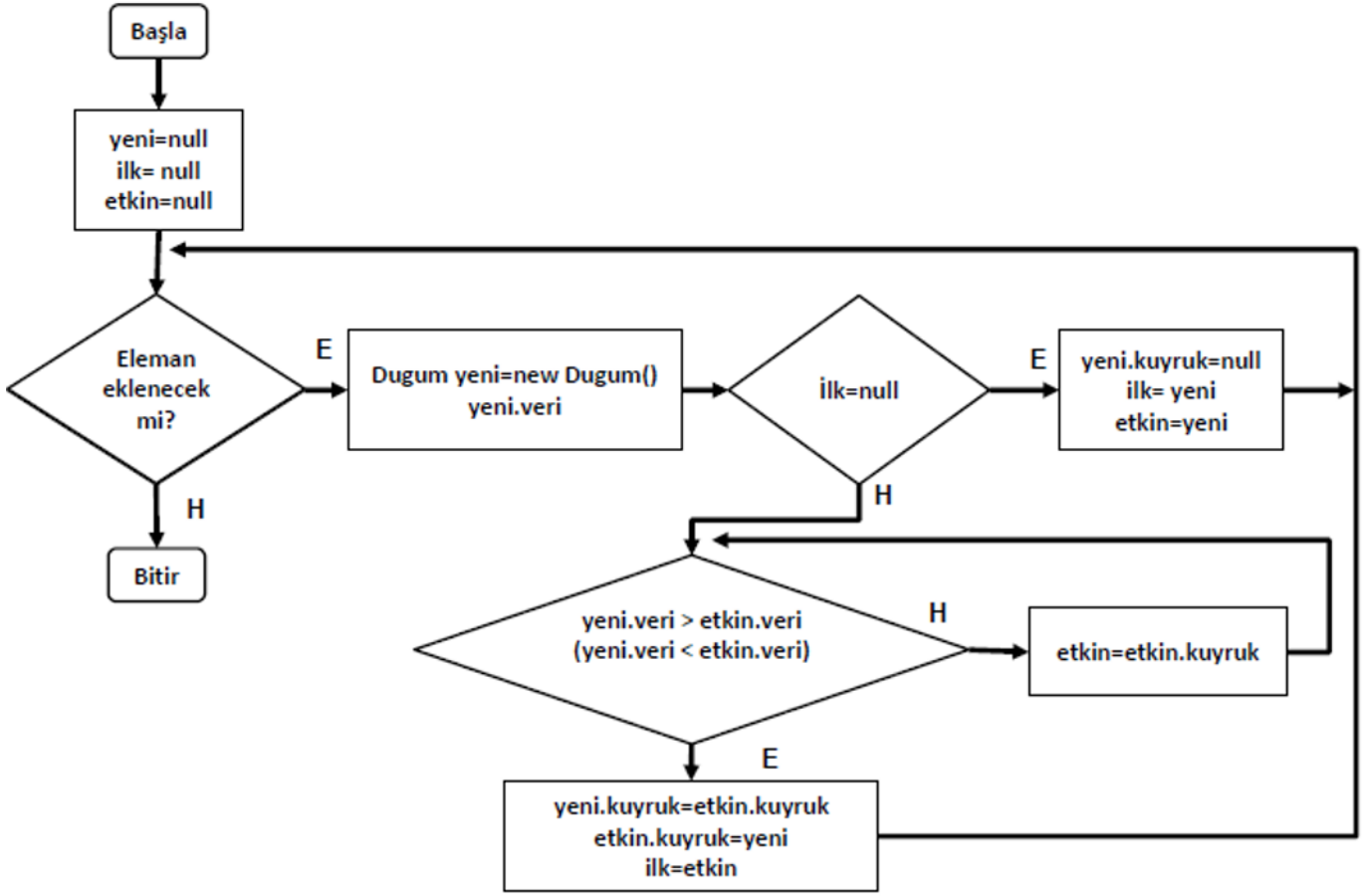
Step 2: Create Node E -> E->data = 4; E->next = F (current).

Step 3: Re-link Node C -> C->next = E (temp).

Final List: Head -> [A | 1] -> [B | 2] -> [C | E] -> [E | F] -> [F | 0]



Insertion at Middle: Algorithmic Flowchart



Basic Operations: 2. Deletion from Beginning

Deletion Scenarios Overview

1. Deletion from Beginning (HEAD) <-- CURRENT FOCUS

2. Deletion from End (TAIL)

3. Deletion from Specific Position (MIDDLE)

Key Principle for Head Deletion:

- Removes the first node and updates Head to the second node.
- Time Complexity: O(1) constant time! (No traversal needed!).

Deleting from Beginning: Algorithm & Diagram

Algorithm Steps

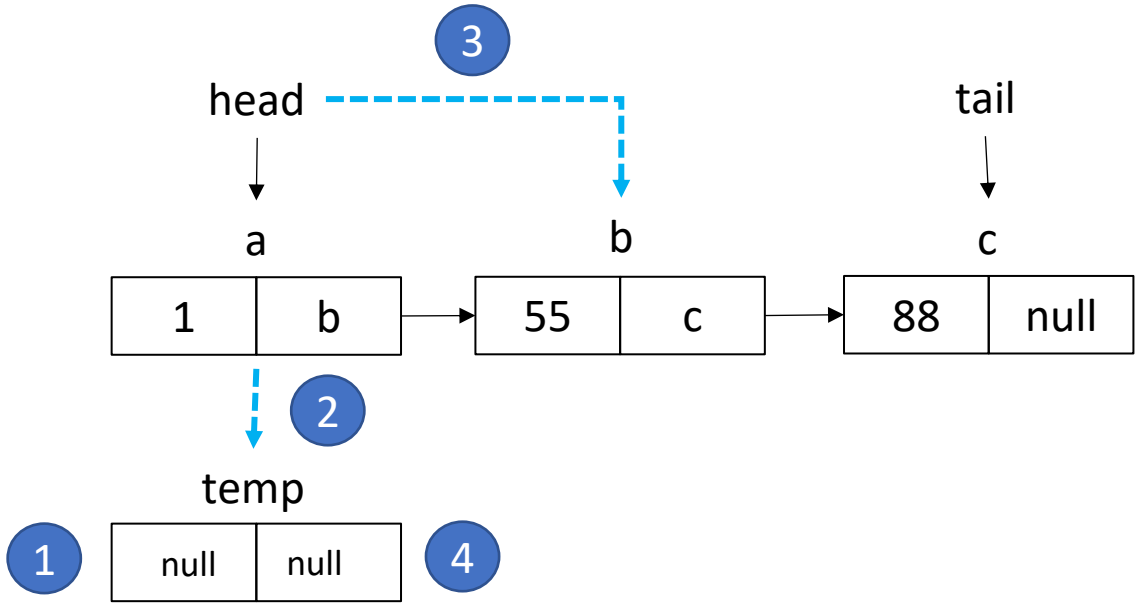
Step 1: Verify list is not empty (head != NULL).

Step 2: Store current head address in temporary pointer: temp = head;

Step 3: Advance head pointer to second node: head = head->next;

Step 4: Free dynamic Heap memory of old head: free(temp);

Step 5: Reset temp = NULL;



Basic Operations: 2. Deletion from End (Tail)

Deletion Scenarios Overview

1. Deletion from Beginning (HEAD)
2. **Deletion from End (TAIL) <-- CURRENT FOCUS**
3. Deletion from Specific Position (MIDDLE)

Key Principle for Tail Deletion:

- Must traverse to the SECOND-TO-LAST node to set its next pointer to NULL.
- Time Complexity: O(N) linear time in singly linked lists!

Deleting from Tail: Algorithm & Diagram

Algorithm Steps

Step 1: Check if list is empty or has only 1 node.

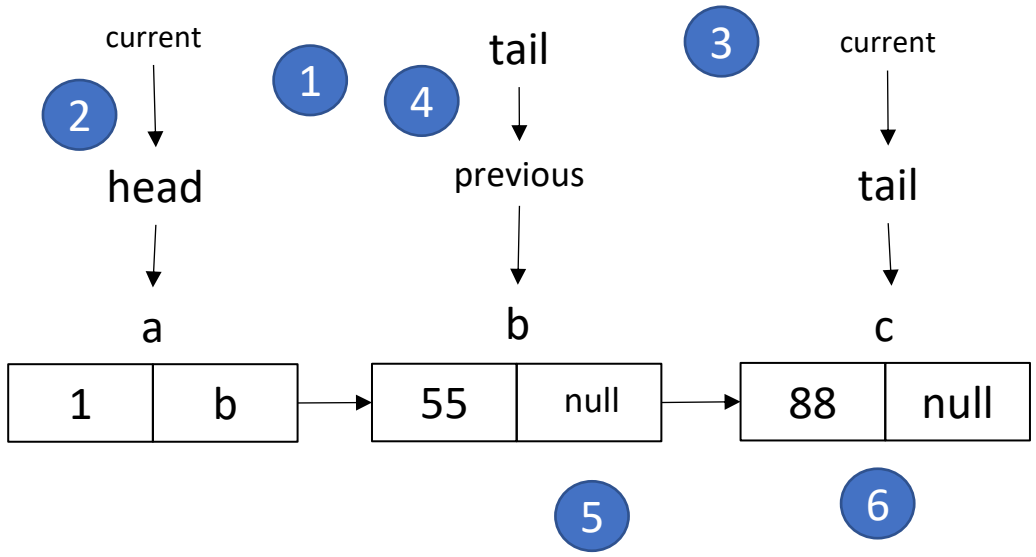
Step 2: Traverse list using 'current' until 'current->next == tail'.

Step 3: Save 'previous' node (second-to-last).

Step 4: Set previous->next = NULL;

Step 5: Free memory of old tail: free(tail);

Step 6: Update tail = previous;



Basic Operations: 2. Deletion from Specific Position

Deletion Scenarios Overview

1. Deletion from Beginning (HEAD)
2. Deletion from End (TAIL)
3. **Deletion from Specific Position (MIDDLE) <-- CURRENT FOCUS**

Key Principle for Middle Deletion:

- Locates target node, bridges 'previous' node directly to 'current->next', and frees target node.
- Time Complexity: O(N) search traversal time.

Deleting Middle Node 'C': Diagram & Steps

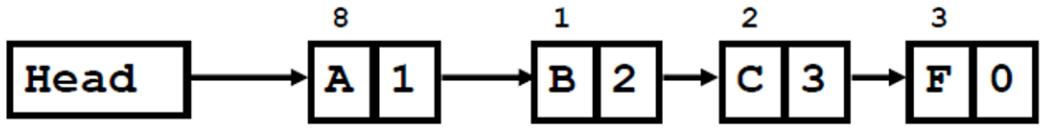
Bypassing Node 'C' in Memory

Initial Chain: Head -> [A] -> [B] -> [C (Target)] -> [F]

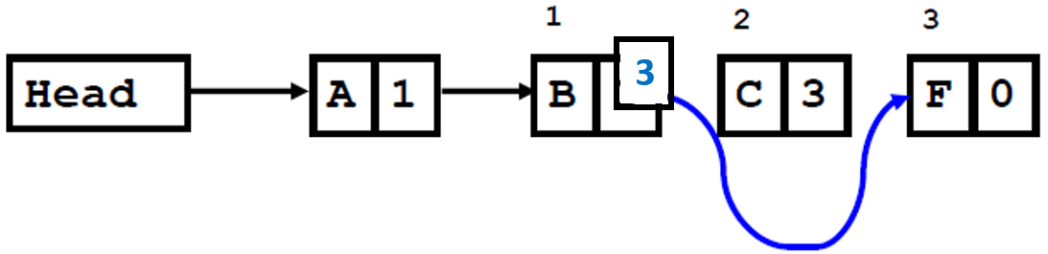
Steps to Remove Node C:

1. Traverse -> previous points to B, current points to C.
2. Re-link B to F -> B->next = C->next (which is F!).
3. Free C -> free(C);

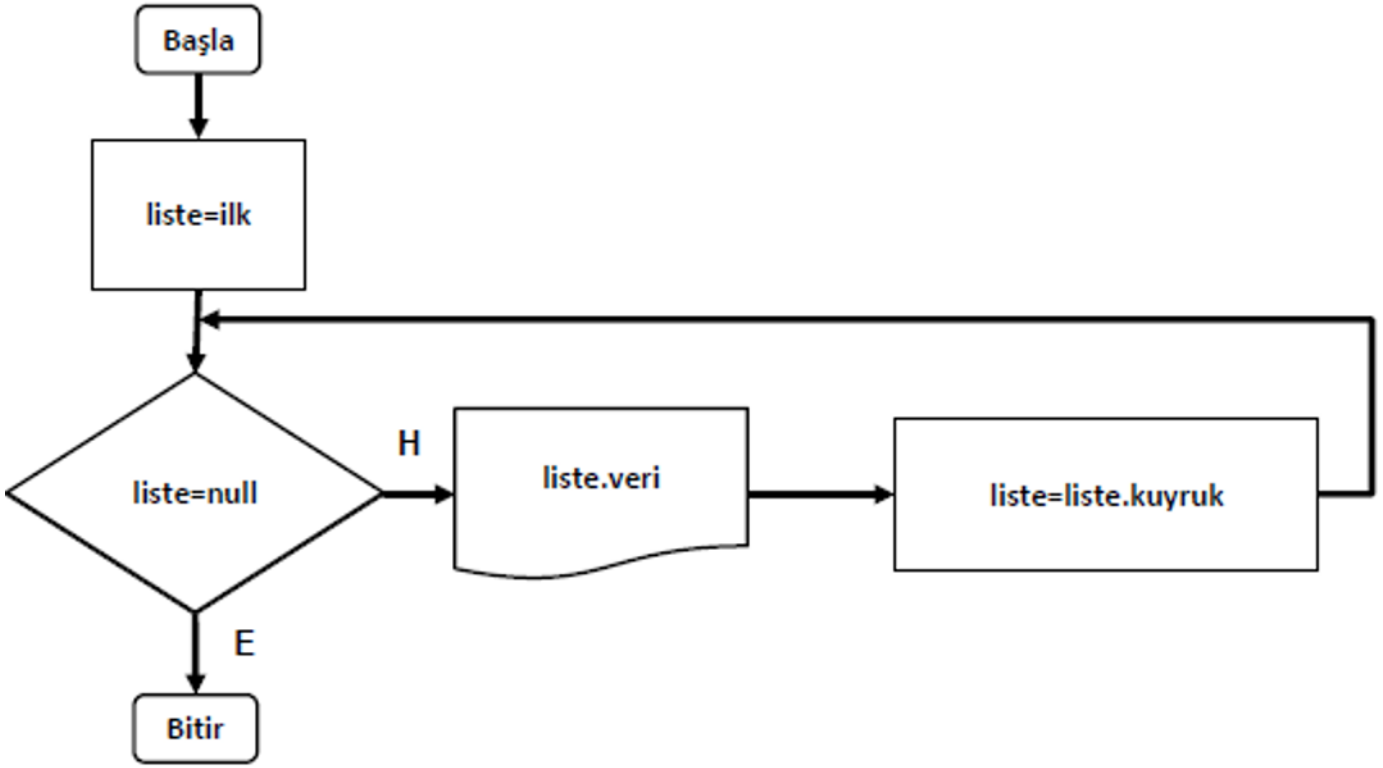
Final Chain: Head -> [A] -> [B] ----------> [F]



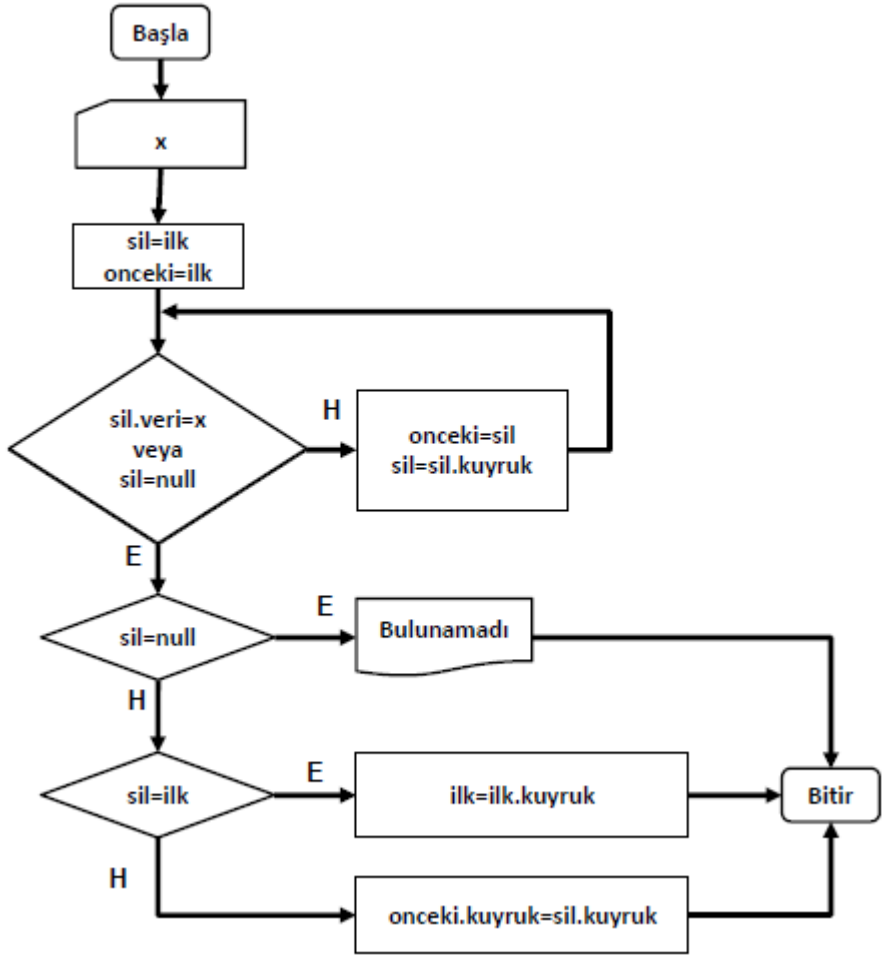
Remove the node that stores "C"



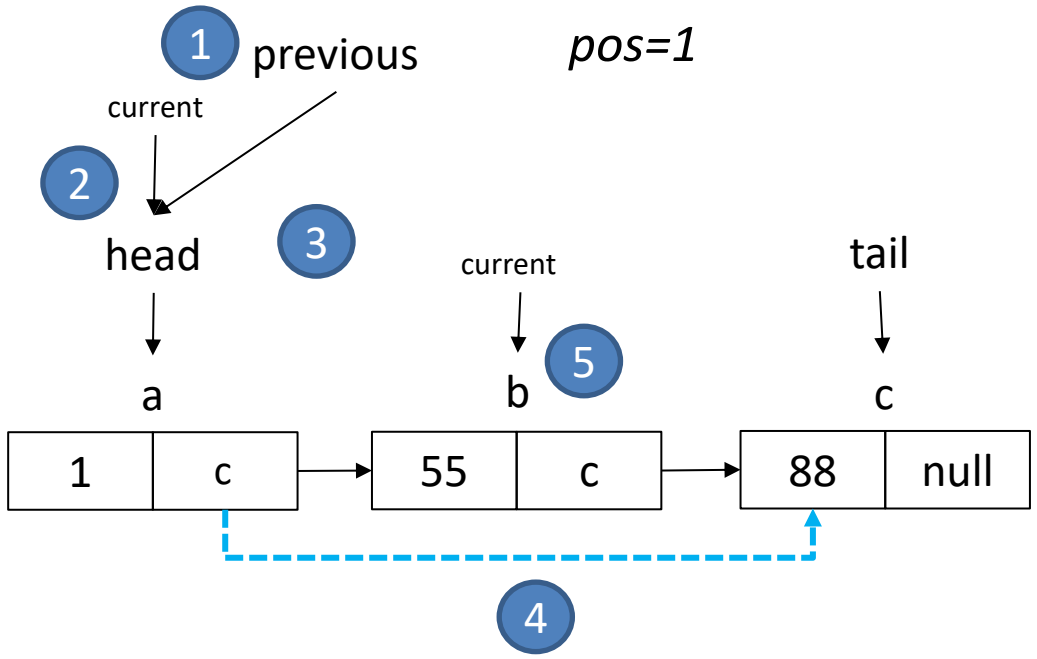
Traversal & Print Flowchart



Deletion from Middle Flowchart



Middle Deletion Visual Trace at pos = 1



Basic Operations: 3. Traversal

Traversal Operation Overview

- Definition: Sequentially visiting every node from Head to Tail to inspect or process payload data.
- Time Complexity: O(N) linear time.
- Standard Loop Pattern:

```
Node *current = head;
while (current != NULL) {
    process(current->data);
    current = current->next;
}
```

Listing All Elements via Traversal

Printing Node Chain

Given List: head -> [1 | b] -> [55 | c] -> [88 | null] (tail)

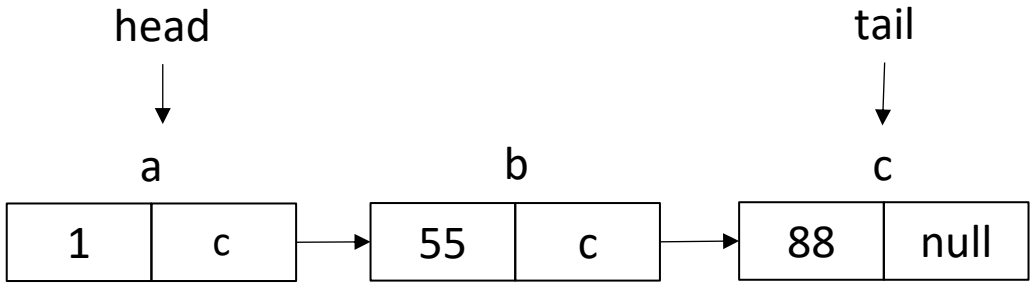
Traversal Trace:

Visit Node 1 -> Print 1

Visit Node 55 -> Print 55

Visit Node 88 -> Print 88

Printed Output Sequence: 1 -> 55 -> 88



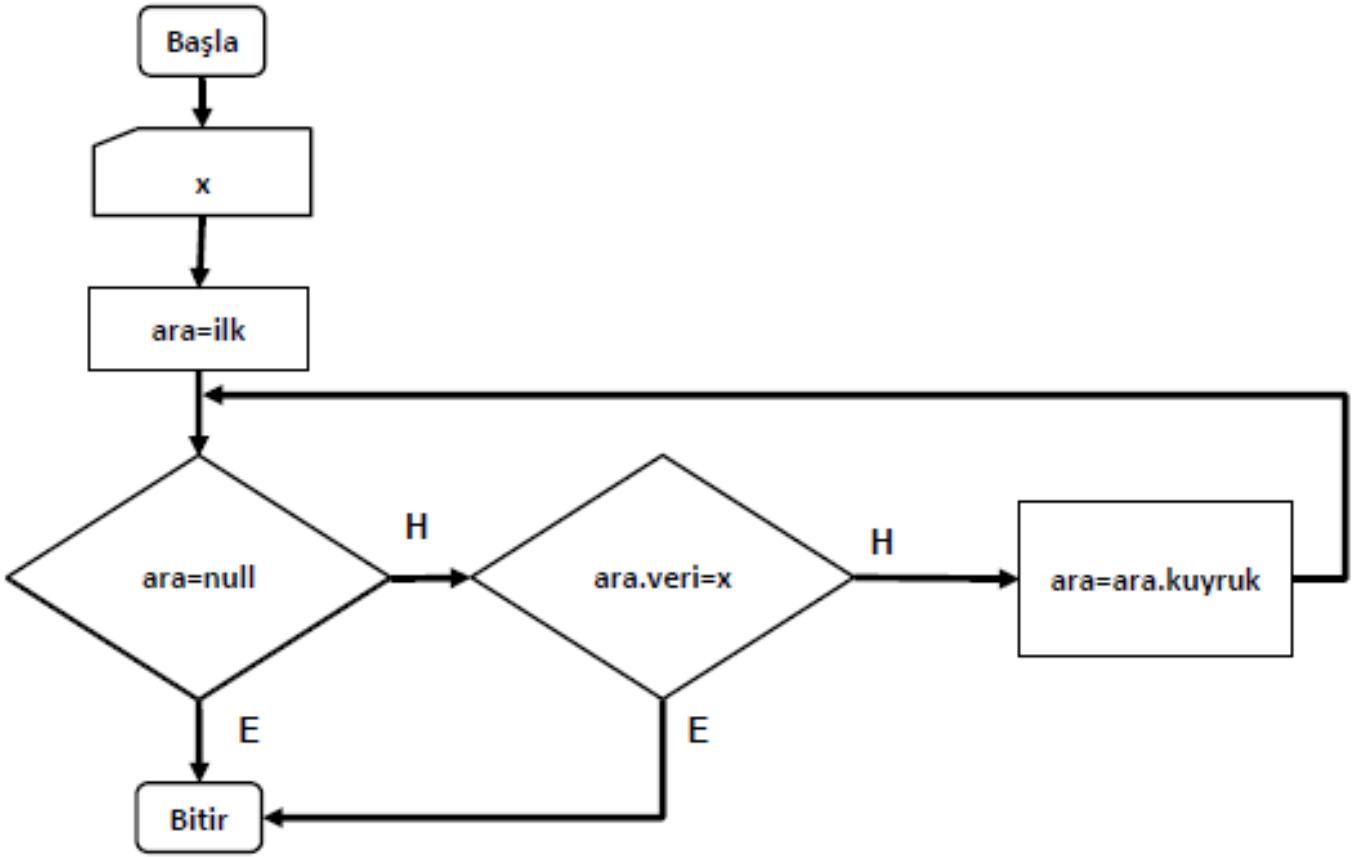
List Nodes: 1 -> 55 -> 88

Basic Operations: 4. Searching

Searching Operation Overview

- Linear Search Strategy: Compares target value 'x' sequentially against node data.
- Best-Case Time: O(1) if target is at Head.
- Worst-Case Time: O(N) if target is at Tail or not present.
- Return: Index position counter or Node pointer.

Linear Search Algorithmic Flowchart



Searching Step-by-Step Visual Trace

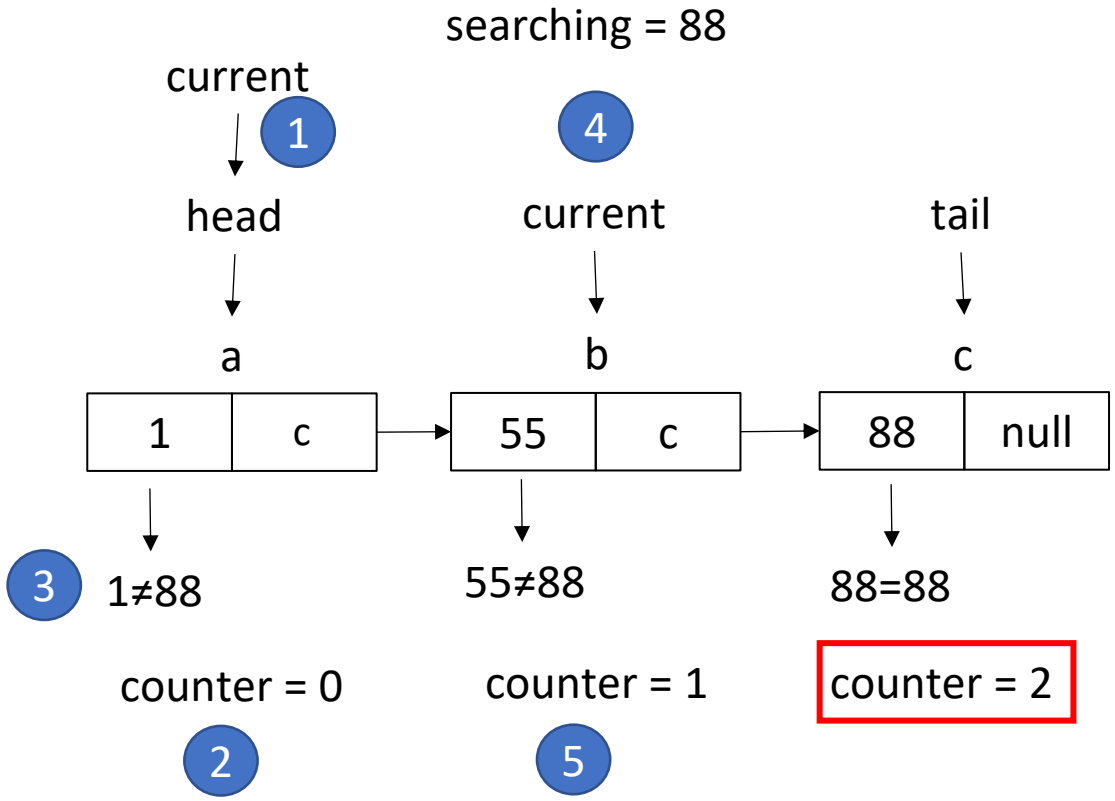
Searching for Value = 88

Given List: head -> [1 | b] -> [55 | c] -> [88 | null]

Search Execution Trace (target = 88):

- Check Node 0 (value 1) : 1 != 88 -> counter = 0, advance current
- Check Node 1 (value 55) : 55 != 88 -> counter = 1, advance current
- Check Node 2 (value 88) : 88 == 88 -> MATCH FOUND at counter = 2!

Result: Target 88 found at index position 2.



Basic Operations: 5. Reversal

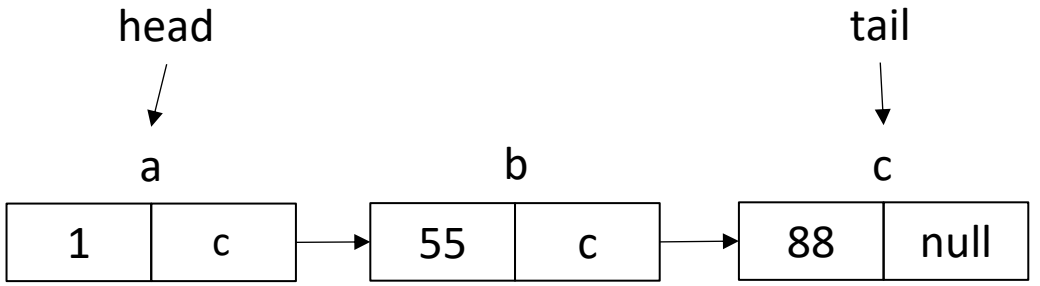
Reversal Operation Overview

- In-Place Reversal: Reverses direction of all next pointers in the list.
- Pointer Tracking: Uses 3 pointers ('previous', 'current', 'next_node').
- Result: Old Tail becomes new Head, old Head becomes new Tail pointing to NULL.
- Time Complexity: O(N) single pass.

Reversal Homework & Exercise

Homework Assignment Problem

- Problem Statement: Write an algorithm/program that prints the elements of a singly linked list in REVERSE order (from Tail to Head).
- Given List: head -> [1] -> [55] -> [88] -> NULL
- Target Output: 88 -> 55 -> 1
- Challenge: Can you do this using Recursion or a Stack data structure without modifying original links?



List nodes: 88 -> 55 -> 1

Overview of Additional Helper Operations

4 Essential ADT Utility Functions

1. Is the list empty? (bos_mu)
2. Delete the entire list (liste_sil / purge)
3. Count the list size (boyut_say)
4. Delete the node storing specified data value (sil)

Additional Op 1: Is List Empty? (`bos_mu`)

Source Code

```
// C Function: Check if Empty
#include <stdbool.h>

bool bos_mu(Node *bas) {
    if (bas == NULL)
        return true;
    else
        return false;
}
```

Is List Full Check?

- Is there a need to check if list is FULL?
- ANSWER: NO!

- Why? Linked lists allocate nodes dynamically on Heap RAM. Unlike static arrays, they have no fixed capacity limit and never become 'full' unless physical system RAM is exhausted!

Additional Op 2: Delete Entire List (`liste_sil`)

Source Code

```
// Proper Deletion of Entire List
void liste_sil(Node **bas) {
    Node *current = *bas;
    Node *next_node;

    while (current != NULL) {
        next_node = current->next;
        free(current); // Free each node!
        current = next_node;
    }
    *bas = NULL; // Reset head to NULL
}
```

Is 'bas = NULL' Enough?

- Question: Is setting 'bas = NULL' enough to delete list?
- Answer: PARTIALLY!
  - YES if you don't care about memory leaks.
  - NO if you want to free allocated Heap RAM!
- Rule: Must traverse entire list node-by-node and free() each individual node to prevent leaks!

Additional Op 3: Count List Size

Size Counting Algorithm

Given List: head -> [1] (count=1) -> [55] (count=2) -> [88] (count=3) -> NULL

Algorithm:

```
 int countNodes(Node *head) {
     int counter = 0;
     Node *current = head;
     while (current != NULL) {
         counter++;
         current = current->next;
     }
     return counter;
 }
```

Result: Returns size = 3 in O(N) time.

Additional Op 4: Delete Node with Specified Data

Source Code

```
// Delete Node by Value
void sil(Node **bas, int deger) {
    int indis = ara(*bas, deger);
    if (indis == -1) {
        printf("Value not found!\n");
    } else {
        if (indis == 0) {
            bas_sil(bas); // Delete head
        } else {
            konumdaki_sil(bas, indis); // Delete middle
        }
    }
}
```

Value-Based Deletion Logic

Logic Steps:

1. Invoke search function 'ara()' to find index of target value.
2. If index == -1 -> Value not present.
3. If index == 0 -> Value is at Head (call bas_sil()).
4. If index > 0 -> Value is in middle (call konumdaki_sil()).

C++ LinkedList Class Implementation (1/4)

Class Declaration & Constructor

```
// C++ Object-Oriented LinkedList Class (Part 1)
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class LinkedList {
private:
    Node* head;
public:
    LinkedList() {
        head = nullptr;
    }
    // Methods continued on next slides...
```

C++ LinkedList Class Implementation (2/4)

Insertion Methods

```
// C++ LinkedList Class: Insertions (Part 2)
    void insertAtBeginning(int value) {
        Node* newNode = new Node();
        newNode->data = value;
        newNode->next = head;
        head = newNode;
    }

    void insertAtEnd(int value) {
        Node* newNode = new Node();
        newNode->data = value; newNode->next = nullptr;
        if (head == nullptr) { head = newNode; return; }
        Node* temp = head;
        while (temp->next != nullptr) temp = temp->next;
        temp->next = newNode;
    }
```

C++ LinkedList Class Implementation (3/4)

Deletion Methods

```
// C++ LinkedList Class: Deletions (Part 3)
    void deleteFromBeginning() {
        if (head == nullptr) return;
        Node* temp = head;
        head = head->next;
        delete temp;
    }

    void deleteFromEnd() {
        if (head == nullptr) return;
        if (head->next == nullptr) { delete head; head = nullptr; return; }
        Node* temp = head;
        while (temp->next->next != nullptr) temp = temp->next;
        delete temp->next;
        temp->next = nullptr;
    }
```

C++ LinkedList Class Implementation (4/4)

Display Method & Main Test Driver

```
// C++ LinkedList Class: Display & Main Driver (Part 4)
    void display() {
        Node* temp = head;
        while (temp != nullptr) { cout << temp->data << " -> "; temp = temp->next; }
        cout << "nullptr" << endl;
    }
};

int main() {
    LinkedList list;
    list.insertAtBeginning(10); list.insertAtEnd(20); list.insertAtEnd(30);
    list.display(); // Output: 10 -> 20 -> 30 -> nullptr
    list.deleteFromBeginning(); list.display(); // Output: 20 -> 30 -> nullptr
    list.deleteFromEnd(); list.display();       // Output: 20 -> nullptr
    return 0;
}
```

Advantages & Disadvantages of Linked Lists

Key Advantages

- **Dynamic Sizing:** Grows/shrinks dynamically at runtime without pre-allocated buffer overhead.
- **Efficient Insertions/Deletions:** O(1) constant-time node insertion and removal without element shifting.
- **Non-Contiguous Allocation:** Utilizes fragmented RAM effectively without requiring unbroken memory blocks.
- **Flexible Topology:** Easily expands into Trees, Graphs, Stacks, and Queues.

Disadvantages & Trade-offs

- **Sequential Access Only:** No O(1) index access; locating element N requires O(N) traversal.
- **Memory Overhead:** Every node incurs extra memory cost for storing pointer links (4-8 bytes).
- **Cache Unfriendly:** Scattered Heap nodes result in lower CPU spatial cache locality compared to contiguous arrays.
- **Complex Debugging:** Risk of dangling pointers, memory leaks, and segmentation faults.

Chapter 7 Summary & Next Steps

Core Concepts Mastered

- Node & Pointer Mechanics: Self-referential structs chaining dynamic Heap RAM nodes.
- Head Anchor: Essential entry pointer to prevent losing entire list.
- CRUD Operations: O(1) Head insertion/deletion vs O(N) search and middle manipulations.
- Memory Safety: Mandatory balance between dynamic allocation and explicit free()/delete.

Upcoming Topics

- Chapter 8: Doubly Linked Lists (Bidirectional prev & next pointers).
- Chapter 9: Circular Linked Lists (Endless ring networks).
- Practical Homework: Complete the Reverse Printing Assignment!